

1 Introduction

Graph representations play a crucial role in many real-world applications, enabling the extraction of meaningful patterns from complex relational data. First of, what is a graph representation in simple terms. A graph is an object with vertices and edges. Machine learning algorithms work with vectors, in supervised learning, you need a label along with a vector. The question you may have is "Is it possible to convert a graph representation of some real world process to a vector so that I can model or study that process with standard machine learning techniques?". The answer is yes, and we will explore one such method in this example. In this example, we focus on constructing a representation for good borrowers within the SBA 7a loans dataset. Such representations are valuable for tasks like building taxonomies of borrower profiles, designing tailored loan products, and identifying borrowers who, despite being classified as good, share similarities with higher-risk individuals thus aiding in risk management.

Previously, we explored feature engineering for borrowers using graph structures. Here, we take a step further by modeling good borrowers as nodes in a graph. We then apply spectral embedding techniques to generate low-dimensional representations of this graph, making the data suitable for downstream machine learning tasks.

2 Methodology

We begin by constructing a graph where each node represents a good borrower. Constructing a graph requires the notion of similarity between borrowers. We define similarity based on distance, shared attributes or some other criteria. For this example, we use a simple distance-based similarity measure. We compute pairwise distances between borrowers using their feature vectors. At this point, the following are the ways to construct a graph:

- *k-nearest neighbors (k-NN)* graph: Connect each borrower to its k -nearest neighbors based on the computed distances.
- *Epsilon-neighborhood* graph: Connect borrowers that are within a certain distance (epsilon) of each other. This is done by selecting a *kernel*, such as the Gaussian kernel, and using it to calculate edge weights. By selecting an appropriate bandwidth for the kernel, we can control the sparsity of the graph.

Note at this point, by simply adjusting the parameters (k or γ , the bandwidth of the kernel), we can partition the graph into different connected

components. This is useful for identifying clusters of similar borrowers. Each connected component can be treated as a subgraph, and we can analyze these subgraphs separately. This is particularly useful for working with large datasets, as it allows us to focus on smaller, more manageable sections of the graph.

Recall that each point in the graph that you constructed above has a set of coordinates (attributes). A question that may arise is "Is the cloud of points associated with the graph on a plane or a curved surface?". If a plane is a reasonable assumption, then, simpler methods such as PCA can yield good low dimensional representations of your points. Manifold learning methods, such as the spectral-embedding can capture non-linear structure as well. For this reason, we will focus on the spectral-embedding technique in this work. A detailed discussion of these techniques is outside the scope of this document, the interested reader can use resources such as [2] for details. The original data representation resided in *ambient space*, spectral embedding determines a low dimensional representation of each data point by preserving the neighborhood around each point. There are a variety of techniques available for manifold learning, see [2] for more information. Another popular way to learn the representation of the graph we just created is by performing a *random walk* on each node of the graph we constructed and then looking the nodes we visit often from each node with a probabilistic perspective and use that to generate the low dimensional embedding, see [4] for the details. Both are popular, reasonable approaches. These are by no means the only ways to learn embeddings on graphs, see [3] and [1] for a comprehensive treatment.

References

- [1] Ines Chami et al. "Machine Learning on Graphs: A Model and Comprehensive Taxonomy". In: *CoRR* abs/2005.03675 (2020). arXiv: [2005.03675](https://arxiv.org/abs/2005.03675). URL: <https://arxiv.org/abs/2005.03675>.
- [2] algorithm Ghodsi. *University of Waterloo, Unsupervised Learning Course Playlist* -- *youtube.com*. <https://www.youtube.com/watch?v=L-pQtGm3VS8&list=PLehuLRPyt1HzQoXEhtNuYTmd0aNQvtyAK>. [Accessed 27-09-2025].
- [3] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.
- [4] Jure Leskovic. *CS 224 Stanford Jure Leskovic Lecture on Random Walk Embeddings* *youtube.com*. <https://www.youtube.com/watch?v=Xv0wRy66Big&t=961s>. [Accessed 27-09-2025].